
What the Final Patch Hides: Event-Level Regression Measurement for Coding-Agent Harnesses

Anonymous Author(s)

Affiliation

Address

email

Abstract

1 Coding agents are evaluated on the patch they leave behind. In a pilot on a
2 40-instance slice of SWE-bench Verified, we show that this final state can hide
3 most of what happens during a run. We built an instrument that records every
4 mutating action of an agent as a git commit on a hidden reference, replays each
5 benchmark instance’s held-out tests at every recorded state inside the benchmark’s
6 own container image, and attributes each detected regression to the harness check
7 that could have caught it. Applied to SWE-bench Verified, the instrument recorded
8 184 regression events across the runs that produced any; the final patch exposed
9 one. Regression frequency depended on the model stack: on the same 40 instances
10 under the same harness, one frontier stack produced no events in 40 runs and
11 another produced 140 events in 11 incidents (Fisher $p = 0.010$). A pre-registered
12 comparison of recovery policies, analysed with code committed before unblinding,
13 found that rollback to a verified checkpoint left contamination in one final tree
14 where no recovery left it in nine (paired sign test, $p = 0.022$), but rollback also
15 resolved fewer tasks, because the verifier that triggers it rejected patches the official
16 grader accepted. Replacing that verifier with the repository’s own tests removed
17 the tradeoff: 65% resolved with no contaminated final tree (McNemar $p = 0.012$
18 against plain rollback), and a variant that reads only half of those tests, graded on
19 the other half, resolved 70% with no held-out failure. We release the instrument,
20 its validation protocol, and a catalogue of 28 measurement defects encountered
21 while building it, 23 of which produced plausible numbers rather than errors.

22 1 Introduction

23 Benchmarks for coding agents grade the final patch. SWE-bench [1] and its Verified subset [2] run
24 the repository’s tests against the submitted patch and report whether the target tests now pass and the
25 previously passing tests still pass. Work on agent-introduced regressions follows the same convention
26 and counts the previously passing tests the submitted patch breaks [8].

27 This convention has a blind spot. A regression that an agent introduces and later repairs leaves no
28 trace in the final patch, and any harness with a recovery mechanism produces that pattern by design.
29 Whether it matters turns on two unanswered questions: how often regressions occur during a run, and
30 how much of that activity survives to the final state.

31 We answer both by measuring the timeline rather than the endpoint, treating observation of the
32 working tree as the operating-system-level primitive on which a recovery primitive should be built.
33 Our contributions are:

1. **An instrument for event-level regression measurement.** Every mutating tool call is recorded as a commit on a git reference the agent cannot see; after the run, the instance’s held-out passing tests are replayed at every recorded state inside the benchmark’s pinned container, and detection is attributed by coverage. Validity is established by controls, a liveness gate, a golden check through the agent’s own execution path, and a re-scoring control (Section 3).
 2. **Measurements on SWE-bench Verified.** Across the GPT-5.6 rollback runs that produced any regression, the timeline recorded 184 events and the final state exposed one (Section 5.2). Event frequency depended on the model stack: identical instances and harness gave 0 events under one frontier stack and 140 under another (Section 5.3).
 3. **A pre-registered comparison of recovery policies, and a fix.** With the analysis committed before unblinding, rollback to a verified checkpoint left fewer contaminated final trees than no recovery ($p = 0.022$) but resolved fewer tasks; we trace the loss to verifier precision, and a post hoc regression-gated verifier removes it, reaching 65% resolve with no contaminated tree, 70% in a split variant graded on tests the gate never reads (Sections 5.4 to 5.6).
 4. **A catalogue of 28 measurement defects** found while building the instrument, classified by mechanism (Appendix A); 23 produced a plausible number rather than an error.
- All measurements are exploratory and were made on a development slice of the benchmark that is excluded from any confirmatory frame. No claim is made about the population of instances beyond that slice.

2 Background and related work

Benchmarks and their validity. SWE-bench [1] grades a patch by tests that must go from failing to passing and tests that must keep passing; Verified [2] is the human-screened 500-instance subset. Concerns about the static subset have accumulated: contamination, memorisation, and leaked solutions [6], flawed tests [7], and rolling [3, 5], private [4], or long-horizon [23] alternatives in response. UTBoost [7] found the held-out tests often insufficient: augmented tests exposed 345 mislabelled patches, affecting 24.4% of Verified leaderboard entries. TDAD [8] measured regressions in submitted patches on Verified (6.1% of runs under a vanilla open-weight agent) and reduced them with a pre-change impact-analysis map that tells the agent which tests to verify; we measure the same quantity on the timeline, and Section 5.6’s gate is the run-the-tests-before-accepting counterpart.

Agent scaffolds and recovery. SWE-agent [9], OpenHands [10], Agentless [11], and mini-swe-agent [12] span the scaffold design space, built on CodeAct [13] and ReAct [14]. Within-episode recovery has been studied as self-reflection [15, 16], progress-gated recovery [17], checkpoint repair [18], and provenance-based rollback [19]; operating-system framings [20] move scheduling, context, memory, and storage management into a kernel for agents, without yet a primitive for observing or restoring the working tree. Process reward models and rubric verifiers [24, 25] approach the verifier-precision problem of Section 5.5 from the reward and test-time-selection side.

Automated program repair. The regression problem is the plausible-versus-correct patch problem of program repair [26, 28, 29]; regression tests are the main defence against overfitting patches [27]. Our results concern regressions the agent’s own process repairs before the patch is tested; trajectory-level evaluation [21, 22] argues that resolve rate alone is uninformative. Public leaderboard trajectories record actions and observations but not the tree at each step, so this measurement cannot be mined from them without re-execution; the instrument commits the tree.

3 Instrument

Figure 1 shows the harness and the instrument. **Observational timeline.** After every mutating tool call, the working tree is committed to a git reference outside the agent’s view, using a private index so that the agent’s own `git status` and `git diff` are unchanged. Rollbacks and the end of the run are observations too, and because observations are commits, an archived run can be re-measured later without re-running the agent.

Exhaustive replay. For each observation, the instance’s passing-test set is run inside the pinned container against that observation’s tree; a regression event is a test that passes at one observation and

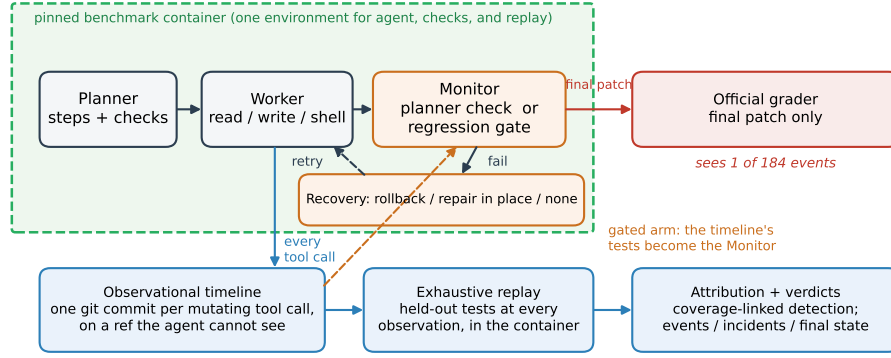


Figure 1: The harness and the instrument. The planner, worker, monitor, and recovery policy run inside the benchmark’s pinned container; every mutating tool call is committed to a hidden git reference; held-out tests are replayed at every observation and detections are attributed by coverage. In the gated arm (dashed), the timeline’s tests serve as the monitor. The official grader sees only the final patch.

85 fails at a later one. Every observation is replayed rather than bisected, because bisection assumes
 86 monotone verdicts and a recovery policy violates that by construction. Replay is scoped to the files
 87 holding the held-out tests, which is what keeps it affordable: 4 to 8 seconds per observation on the
 88 two repository families we timed (pytest and django), about 26 CPU-minutes for a 40-instance sweep,
 89 and a projected 80 CPU-hours for 500 instances at 100 edits each. Infrastructure failures during
 90 replay are recorded as missing observations rather than as test failures, and baseline-dead tests are
 91 excluded from event counting.

92 **Attribution.** A detected regression is classified three ways: *attributed* if a failing harness check
 93 and the broken held-out test both exercise a file the agent changed at that observation (using a
 94 per-instance coverage map built at the base commit), *co-occurring* if some harness check failed while
 95 the regression was open but no such link exists (an over-count of detection), and *unknown* if the
 96 coverage map cannot say.

97 **Execution environment.** The agent’s tools and the harness’s checks execute inside the same pinned
 98 container as the replay, with file changes synchronised to the host tree the timeline records (Appendix
 99 A.2).

100 **Validation.** Five gates run before any paid experiment: a negative control, a positive control with
 101 injected regressions including recovered ones, a flake screen, an unknown-rate ceiling, and a baseline
 102 liveness check. A golden check drives the gold patch through the agent’s real tool path and requires
 103 the grader to return "resolved" (a null run "unresolved"); it passed on three repository families. A
 104 re-scoring control re-measures an archived timeline under a changed instrument with no model calls;
 105 on both runs to date it reproduced the archived episode counts.

106 4 Experimental setup

107 **Benchmark and slice.** SWE-bench Verified, 500 instances. A 40-instance development slice (16
 108 from django), stratified and fixed before any measurement, was used throughout and is excluded from
 109 any confirmatory frame. The GPT-5.6 stack also ran on the slice’s first 10 instances as a calibration,
 110 and plain rollback ran twice.

111 **Harness.** A planner decomposes the task into steps with verification commands; a worker executes
 112 each step with three tools (read, write, shell); a monitor runs the verification, and on failure the
 113 recovery policy acts: **rollback** (reset to the last verified checkpoint, retry with feedback), **repair-**
 114 **in-place** (retry from the failed tree), or **no recovery** (keep the failed step’s tree). Each run has a \$4
 115 work-cost cap; exhaustion is scored as failure.

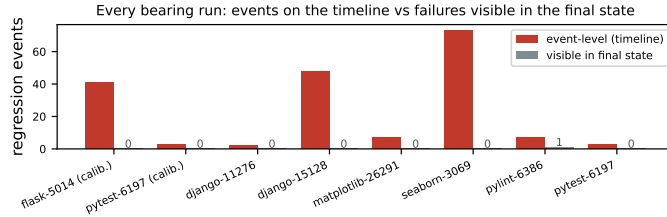


Figure 2: Every run that produced at least one regression event, in both GPT-5.6 sweeps. Left bars (red) are events recorded on the timeline; right bars (grey) are held-out test failures visible in the graded final patch.

Models. Two frontier stacks under identical harness, policy, instances, and caps, named by their API model strings: `claude-opus-4-7` (planner) with `claude-sonnet-4-6` (worker), and `gpt-5.6-sol` (planner) with `gpt-5.6-terra` (worker); identifiers are recorded in every run manifest.

Outcomes. Resolve is the official grader’s verdict on the final patch. Events are reported at three levels because the distribution is overdispersed: distinct incidents (observations at which at least one test broke), declared events (one per test function and onset, parametrised variants collapsed), and bearing runs. Final-state contamination is the number of held-out tests failing in the graded final patch, net of baseline-dead tests.

Pre-declaration. The event unit, the gates, and the contrast were declared before the relevant data existed; the contrast’s analysis (exact paired sign tests, final-state contamination primary, incident exposure co-primary) was committed before either comparison arm finished. One amendment, making the gates conditional on the model stack, preceded unblinding (Appendix B).

5 Results

5.1 Resolve and cost

Under rollback, the Claude stack resolved 13 of 40 instances (32.5%, exact 95% CI 18.6% to 49.1%) for a total sweep cost of \$34.24; the GPT-5.6 stack resolved 13 of 40 (32.5%; 13 of the 35 graded, 37.1%) for \$8.14. Three GPT-5.6 cells were never attempted (circuit breaker) and two were lost to a file-synchronisation defect; all five count as unresolved out of 40 and are excluded from the graded denominator. One instance cannot be resolved by any arm under the current official parser (Appendix A.3), so every rate has a ceiling of 39. Runs are short: the median run makes 5 mutating tool calls (IQR 4 to 8, maximum 19 over 235 runs; 94% make 10 or fewer), an order of magnitude below public-scaffold trajectories. Paired comparisons use only instances graded in both arms.

5.2 The final state hides almost all regression activity

Figure 2 shows every run that produced a regression event under the GPT-5.6 stack with the rollback policy, pooling the 10-instance calibration and the 40-instance sweep (47 runs). The timeline recorded 184 declared events across eight runs; the final patch exposed one. Run-weighted, seven of the eight bearing runs ended with a clean final patch; three storms (73, 48, and 41 events) supply 88% of the event count, so the run-weighted figure is the more robust statement.

Figure 3 shows a single run in detail. The grader marked it resolved with all graded tests passing; its timeline contains three regressions of the same test function at observations 3, 5, and 7, each repaired by rollback. Two of the three went undetected by any harness check while open. Across the full GPT-5.6 sweep, 48 of 162 raw episodes had no co-occurring harness failure at all.

5.3 Regression frequency depends on the model stack, not the benchmark

On the same 40 instances, under the same harness and rollback policy, the Claude stack produced no regression events in 40 runs (227 observations, all replayed). The GPT-5.6 stack produced 140

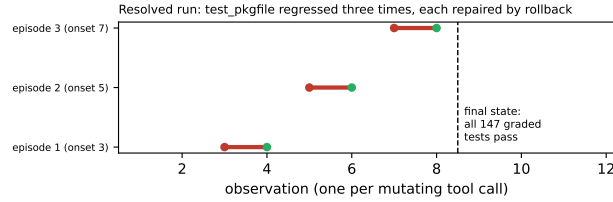


Figure 3: One run that the official grader marked resolved, with all 147 graded tests passing. The timeline shows three regressions of the same test function, each repaired by rollback two observations later.

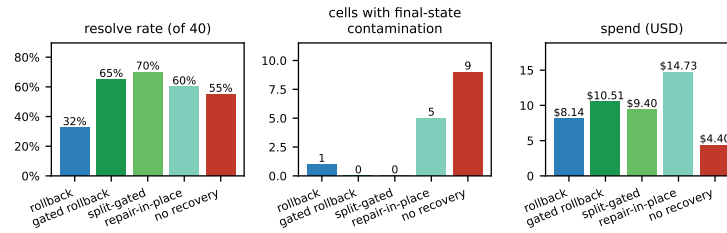


Figure 4: The five recovery arms under the GPT-5.6 stack on the same 40 instances. Contamination counts cells whose final patch fails a previously-passing test.

152 declared events in 11 incidents across 6 of 37 attempted runs (bearing fraction 16.2%, exact CI 6.2%
 153 to 32.0%). A one-sided Fisher exact test on the bearing fractions gives $p = 0.010$; this is a single
 154 pairwise test on six bearing runs (reassign three of them and $p = 0.11$), and the two stacks also traverse
 155 different provider adapters. Observation density differs by a factor of 1.2, which does not account for
 156 140 to 0. The two stacks resolved the same number of instances (13 of 40 each) for total sweep costs
 157 of \$34.24 against \$8.14: the stack that never broke adjacent behaviour cost four times as much per
 158 run. The three storms that supply 88% of the events are model edits, not harness artifacts: a six-line
 159 change to seaborn’s `plot.py`, a nine-line change to django’s `query.py`, and a 262-line deletion in
 160 flask’s `blueprints.py`, none truncated, none containing a shell idiom, and none from a capped turn
 161 (defect D10 was fixed before these sweeps).

162 The Claude zero is measured, not a detection failure: the same stack’s earlier canary run produced
 163 three events, and re-scoring its archived timeline under the sweep’s instrument reproduces them.

164 We had first read the low event rate as a property of the benchmark; the second stack falsified that.
 165 The pre-declared event-rate gate still failed for both stacks, so no arm here is a confirmatory pass
 166 (Appendix B), and we claim no more than a single-sweep dependence on the model stack.

167 5.4 Recovery policy: rollback keeps the final tree clean

168 Figure 4 summarises the pre-declared contrast. The primary endpoint, final-state contamination, is
 169 paired by instance: no recovery was worse than rollback on 9 instances, better on 1, and tied on
 170 25 (exact sign test, $p = 0.022$). Repair-in-place was worse on 5, better on 1, and tied on 29 ($p =$
 171 0.22). The co-primary endpoint, incident exposure, did not differ detectably for either comparison
 172 (no recovery $p = 0.45$; repair-in-place $p = 1.0$): regressions occurred at similar rates under every
 173 policy, and the policy determined whether they persisted.

174 **Disclosure.** The first unblinding gave $p = 0.375$. Seven cells (five no-recovery, two repair-in-place)
 175 whose final trees made the suite uncollectable had been dropped as ungradable, removing the most
 176 contaminated states from the endpoint they bore on most, whereas official grading scores such a
 177 patch as failing every test. The grader was corrected to distinguish an environment failure from a
 178 patch that kills the suite, and the seven archived trees were re-graded without re-running any agent,
 179 giving $p = 0.022$. The rule changed after the data were seen (Appendix B).

5.5 Rollback loses resolution to verifier precision

Rollback resolved fewer tasks than either alternative (Figure 4, left): 13 of 40 against 24 of 40 for repair-in-place and 22 of 40 for no recovery; paired by instance, it won one discordant pair and lost nine against repair-in-place (McNemar exact $p = 0.022$) and won three and lost nine against no recovery ($p = 0.15$). The loss is attributable to the verifier: of the 18 rollback runs that failed, 15 failed at the first step, and 9 of the 18 were resolved under a policy that kept the rejected work; the monitor’s planner-written check had rejected a patch the grader accepted, and rollback discarded it.

5.6 Regression-gated rollback removes the tradeoff

A fourth arm, added after the contrast was unblinded and therefore exploratory, replaces the monitor’s planner-written check with the repository’s previously-passing tests, run in the agent’s container at the start and after every attempt; a step is rejected only if a test that passed at the start fails now. It resolved **26 of 40 instances (65.0%)**, the most of any arm, with **no contaminated final tree** (Figure 4). Paired against plain rollback on the 35 shared instances, it resolved 10 that plain rollback did not and lost 1 (McNemar exact $p = 0.012$; post hoc, one of eight tests reported, unadjusted; paired onset exposure $p = 0.73$). The gate reads the instance’s previously-passing tests, selected from benchmark metadata, so its baseline oracle coincides with the grading oracle (median ratio 1.00) and the clean final trees are guaranteed by construction. Both gated arms also *select* their tests from that metadata, which observes roughly the gold test patch’s blast radius, information a deployed agent lacks; the deployed analogue gates on the full repository suite at baseline (django’s takes about two minutes per attempt, against 4 to 8 seconds scoped, with 97 baseline failures and errors a flake screen must absorb) or on a metadata-free selector such as TDAD’s dependency map [8], and the gated resolve rates are upper bounds on either. A fifth arm, a split variant, removes that guarantee: the gate reads results for a deterministic half of the previously-passing test ids (2,278) and never reads the other half (2,585), on which the grader then rules. It resolved **28 of 40 (70.0%)**, indistinguishable from the full gate (5 discordant pairs against 3, $p = 0.73$) and 13 pairs against 2 over plain rollback (McNemar exact $p = 0.007$), with no final tree failing a held-out test, net of the parser-capped instance of Section 5.1. A second plain-rollback sweep reproduced the first (13 and 13 resolved; 6 of 37 bearing both times), so the gain is outside run-to-run variability. Running the tests before accepting an edit is standard advice; the contribution is measuring what it buys and why plain rollback loses it.

6 Discussion

Why timeline regressions matter. A regression the agent repairs before submission did not reach the user, but it is not free: across the bearing runs, 27% of spend (\$0.63 of \$2.33 summed over the eight bearing runs) went to work done while a regression was open. The events are the per-step ground truth that process reward models for software agents [24, 25] need; for agent-OS design, a recovery primitive without an observation primitive hides exactly the activity that distinguishes recovery policies.

Summary. Final-state evaluation misses the regressions an agent’s own process repairs, nearly all of them in our measurements; measuring the timeline is feasible, validates against controls, separates recovery policies the final patch cannot, and locates rollback’s cost in verifier precision. Building the instrument produced 28 defects (Appendix A); two rules, record infrastructure failures as missing observations and require a positive liveness signal, would have prevented most of them.

7 Limitations

All measurements come from a 40-instance development slice, one sweep per arm, with variability measured only for plain rollback under GPT-5.6. The cross-stack comparison is six bearing runs, confounded with the provider adapter; the gated arms are post hoc and select their tests from benchmark metadata. Runs are an order of magnitude shorter than public-scaffold trajectories (median 5 mutating calls against tens to hundreds), so recovery dynamics at length are unmeasured. The held-out tests observe roughly the gold test patch’s blast radius, so event counts are lower bounds; intervals ignore repository clustering; and the instrument has not been applied to a public scaffold, which is the next experiment.

References

- [1] C. Jimenez et al. SWE-bench: Can language models resolve real-world GitHub issues? ICLR 2024. arXiv:2310.06770.
- [2] OpenAI. Introducing SWE-bench Verified. OpenAI blog, August 2024. openai.com/index/introducing-swe-bench-verified.
- [3] L. Zhang et al. SWE-bench Goes Live! NeurIPS 2025 Datasets and Benchmarks. arXiv:2505.23419.
- [4] X. Deng et al. SWE-Bench Pro: Can AI agents solve long-horizon software engineering tasks? arXiv:2509.16941, 2025.
- [5] I. Badertdinov et al. SWE-rebench: An automated pipeline for task collection and decontaminated evaluation of software engineering agents. NeurIPS 2025. arXiv:2505.20411.
- [6] S. Liang et al. The SWE-Bench Illusion: When state-of-the-art LLMs remember instead of reason. arXiv:2506.12286, 2025.
- [7] B. Yu et al. UTBoost: Rigorous evaluation of coding agents on SWE-Bench. ACL 2025. arXiv:2506.09289.
- [8] P. Alonso, S. Yovine, and V. A. Braberman. TDAD: Test-driven agentic development: Reducing code regressions in AI coding agents via graph-based impact analysis. arXiv:2603.17973, 2026.
- [9] J. Yang et al. SWE-agent: Agent-computer interfaces enable automated software engineering. NeurIPS 2024. arXiv:2405.15793.
- [10] X. Wang et al. OpenHands: An open platform for AI software developers as generalist agents. ICLR 2025. arXiv:2407.16741.
- [11] C. S. Xia et al. Agentless: Demystifying LLM-based software engineering agents. FSE 2025. arXiv:2407.01489.
- [12] K. Lieret and C. E. Jimenez. mini-swe-agent. Software, 2025. github.com/SWE-agent/mini-swe-agent.
- [13] X. Wang et al. Executable code actions elicit better LLM agents. ICML 2024. arXiv:2402.01030.
- [14] S. Yao et al. ReAct: Synergizing reasoning and acting in language models. ICLR 2023. arXiv:2210.03629.
- [15] N. Shinn et al. Reflexion: Language agents with verbal reinforcement learning. NeurIPS 2023. arXiv:2303.11366.
- [16] A. Madaan et al. Self-Refine: Iterative refinement with self-feedback. NeurIPS 2023. arXiv:2303.17651.
- [17] A. Kadu and A. Krishnan. ReflexGrad: Within-episode failure recovery in LLM agents via progress-gated dual-process routing. arXiv:2511.14584, 2025.
- [18] P. Mazaheri. REPOT: Recoverable program-of-thought via checkpoint repair. arXiv:2605.30052, 2026.
- [19] Y. Wang et al. From agent traces to trust: A survey of evidence tracing and execution provenance in LLM agents. arXiv:2606.04990, 2026.
- [20] K. Mei et al. AIOS: LLM agent operating system. COLM 2025. arXiv:2403.16971.
- [21] S. Kapoor et al. Holistic Agent Leaderboard: The missing infrastructure for AI agent evaluation. arXiv:2510.11977, 2025.
- [22] R. Shu et al. What resolve rate hides: Trajectory structure diagnostics for coding agents. arXiv:2607.06184, 2026.

- [23] T. Le et al. SWE-EVO: Benchmarking coding agents in long-horizon software evolution scenarios. arXiv:2512.18470, 2025.
- [24] M. Raghavendra et al. Agentic rubrics as contextual verifiers for SWE agents. ACL 2026. arXiv:2601.04171.
- [25] M. L. Dihan and M. A. R. Khan. SWE-Shepherd: Advancing PRMs for reinforcing code agents. arXiv:2604.10493, 2026.
- [26] Z. Qi, F. Long, S. Achour, and M. Rinard. An analysis of patch plausibility and correctness for generate-and-validate patch generation systems. ISSTA 2015. doi:10.1145/2771783.2771791.
- [27] Y. Lou et al. When automated program repair meets regression testing: An extensive study on two million patches. ACM TOSEM 33(7), 2024. arXiv:2105.07311.
- [28] Z. Fei et al. Patch correctness assessment: A survey. ACM TOSEM 34(2), 2025. doi:10.1145/3702972.
- [29] H. Ye, M. Martinez, and M. Monperrus. Automated patch assessment for program repair at scale. Empirical Software Engineering 26(2), 2021. doi:10.1007/s10664-020-09920-w.
- [30] SWE-bench issue #601: Test result hijacking via stdout forging in evaluation harness. github.com/SWE-bench/SWE-bench/issues/601, June 2026.
- [31] OpenHands issues #4235 (October 2024) and #7044 (March 2025): Docker and environment errors in SWE-bench instance evaluation. github.com/OpenHands/OpenHands.
- [32] J. Yang et al. SWE-smith: Scaling data for software engineering agents. NeurIPS 2025 Datasets and Benchmarks. arXiv:2504.21798.
- [33] J. Pan et al. Training software engineering agents and verifiers with SWE-Gym. ICML 2025. arXiv:2412.21139.
- [34] S. Liu et al. Context as a tool: Context management for long-horizon SWE-agents. Findings of ACL 2026. arXiv:2512.22087.

A The measurement defect catalogue

Two rules would have prevented most of the defects below: record infrastructure failures as missing observations, and require a positive liveness signal.

A.1 The catalogue by mechanism

Each row is a defect found while building the instrument. **S**: it produced a plausible number rather than an error. **G**: it was present while the test suite passed (A4 is the suite). **H**: it was findable only on a clean host. Class A rows depend on the machine the code runs on; class B rows render an infrastructure failure as a measurement; class C rows measure a state where an event was defined; class D rows consume a producer’s output without checking the producer.

#	Defect	Consequence	S	G	H
A1	Shadow commits inherited the machine’s git identity	timeline silently empty on any clean machine	✓	✓	✓
A2	Gate probe invoked bare <code>python</code>	every probe exit 127 on a clean host	✗	✓	✓
A3	Unpinned SDK resolved a different major version	two machines ran different code	✗	✓	✓
A4	Test fixtures verified with bare <code>pytest</code> from PATH	15 tests fail on a clean host as harness defects	✗	—	✓
A5	A benchmark scorer invoked bare <code>python</code>	score 0.0 from a missing interpreter	✓	✓	✓
A6	Agent executed on the host; measurement in the container	26 of 28 zero-step runs; see A.2	✓	✓	✓
B1	Output marker used shell :	a 13/13 run scored as 13 errors	✓	✓	
B2	Results on stderr, markers on stdout	one framework’s results outside the parsed slice	✓	✓	
B3	Probe diffed against a deleted commit	every graded test a hole	✓	✓	
B4	Interrupted mirror clone accepted with zero commits	later instances fail far from the cause	✓	✓	
B5	Container workdir unvalidated	every exec exit 127	✓	✓	
B6	Isolation removed loopback, not only the internet	23.5% of the oracle dead	✓	✓	
306 B7	Provider cache served removed containers	later cells replay against nothing and score clean	✓	✓	
B8	Tree reset reverted image build-time edits	one repository family’s oracle dead	✓	✓	
C1	Bisection assumed monotone verdicts	recovered regressions invisible	✓	✓	
C2	Declared event unit not implemented	rate off by up to $2.7\times$	✓	✓	
C3	Rollbacks produced no observation	recoveries invisible to the timeline	✓	✓	
C4	"Persists past the step boundary" undefined	one reading excludes the events of interest	✓	✓	
D1	Audit field never populated	0 tool errors reported, always	✓	✓	
D2	Malformed planner output crashed the run	33% of runs lost as task failures	✗	✓	
D3	Dependency install counted as agent work	attribution vacuous	✓	✓	
D4	Join took the first observation, not the last	failures pinned to the wrong tree	✓	✓	
D5	Executed config rebuilt from a name	manifest described a different run	✓	✓	
D6	Git handles never released	sweeps die after ~400 cells	✓	✓	
D7	Console re-walked the tree per run	presented as a dead server	✗	✓	
D8	Detection events lacked the ids attribution joined on	attributed detection always unknown	✓	✓	
D9	Absent coverage rendered as measured silence	unmeasured episodes reported as silent	✓	✓	
D10	Output-capped turns executed	half-written file; a 78-event storm	✓	✓	

307 Totals: 23 of 28 silent; 27 of 28 present under a passing suite; 6 of 28 findable only on a clean host.
308 Seven further defects found after this census closed are described where they arose (Section 5.4,
309 Appendix A.3, and the repository log).

310 A.2 The defect that invalidated a conclusion

311 After nineteen fixes, every validation gate passed, the re-scoring control reproduced archived episode
312 counts exactly, and three pilots (80 runs) measured an event rate far below the pre-declared gate.
313 We drafted the conclusion the rule directed: the benchmark offered too little opportunity, so switch
314 benchmarks. The conclusion was wrong. The harness ran the agent on the host in a bare source
315 checkout, while every probe and every gate ran inside the pinned container. On the host, `import`
316 `matplotlib` in that checkout succeeds by importing the uncompiled source tree as a namespace
317 package, and everything downstream fails in ways indistinguishable from an incompetent agent.
318 Twenty-six of 28 zero-step runs traced to this. The gates had validated the measurement path and
319 never the agent’s execution path. The fix was to route the agent’s tools and checks through the same
320 container as the replay, and to add a per-cell environment parity check that runs before any model
321 call.

322 A.3 Mechanisms in public infrastructure

323 Class A: OpenHands issues #4235 and #7044 [31], environment construction failures against SWE-
324 bench images. Class B: UTBoost’s finding that the held-out oracle is insufficient at leaderboard scale
325 [7], and, on SWE-bench Live, baseline tests that fail in the unmodified image because the calendar
326 passed a deprecation date encoded in the instance. Class C: final-state regression measurement [8].
327 Class D: the SWE-bench grader accepting forged test output on stdout [30], and, at the harness’s
328 current revision, a parser that keys parametrised ids by their full text while the dataset stores them
329 truncated at the first space (the earlier parser’s behaviour), so 16 previously-passing ids of one instance
330 in our slice match no output and every submission to it grades as unresolved.

331 **B Pre-declaration and analysis**

332 The event unit is one (test function, onset observation) pair with parametrised variants collapsed. The
333 gates for proceeding on a substrate were an event rate of at least 0.30 per run and a bearing fraction
334 of at least 25%. Both stacks failed the conjunction (bearing 0% and 16.2%); the gates were then
335 re-declared per (substrate, model stack) before the contrast was unblinded, and no arm reported here
336 is confirmatory. The contrast's primary endpoint is final-state contamination, paired by instance, exact
337 two-sided sign test; the co-primary is incident exposure. The analysis script was committed before
338 either comparison arm finished. The regression-gated arm was added after the contrast was unblinded
339 and is exploratory. No multiplicity adjustment was planned or applied. Budget exhaustion is scored
340 as failure. Instances whose baseline oracle records failures in the unmodified image are excluded
341 from resolve-rate comparability claims, and their dead tests are typed as missing observations for
342 event counting.

343 **Grading correction between unblindings.** The first unblinding of the recovery-policy contrast
344 gave $p = 0.375$ for the primary endpoint: seven cells (five no-recovery, two repair-in-place) had been
345 dropped as ungradable because their final trees made the suite uncollectable. Official grading scores
346 such a patch as failing every test, so the most contaminated states had been dropped from the endpoint
347 they bore on most. The grader now distinguishes an environment failure from a patch that kills the
348 suite, and the seven archived trees were re-graded without re-running any agent.